

Accessing Text Corpora and Lexical Resources

Practical work in Natural Language Processing typically uses large bodies of linguistic data, or **corpora**. The goal of this chapter is to answer the following questions:

1. What are some useful text corpora and lexical resources, and how can we access them with Python?
2. Which Python constructs are most helpful for this work?
3. How do we avoid repeating ourselves when writing Python code?

This chapter continues to present programming concepts by example, in the context of a linguistic processing task. We will wait until later before exploring each Python construct systematically. Don't worry if you see an example that contains something unfamiliar; simply try it out and see what it does, and—if you're game—modify it by substituting some part of the code with a different text or word. This way you will associate a task with a programming idiom, and learn the hows and whys later.

2.1 Accessing Text Corpora

As just mentioned, a text corpus is a large body of text. Many corpora are designed to contain a careful balance of material in one or more genres. We examined some small text collections in [Chapter 1](#), such as the speeches known as the US Presidential Inaugural Addresses. This particular corpus actually contains dozens of individual texts—one per address—but for convenience we glued them end-to-end and treated them as a single text. [Chapter 1](#) also used various predefined texts that we accessed by typing `from book import *`. However, since we want to be able to work with other texts, this section examines a variety of text corpora. We'll see how to select individual texts, and how to work with them.

Gutenberg Corpus

NLTK includes a small selection of texts from the Project Gutenberg electronic text archive, which contains some 25,000 free electronic books, hosted at <http://www.gutenberg.org/>. We begin by getting the Python interpreter to load the NLTK package, then ask to see `nltk.corpus.gutenberg.fileids()`, the file identifiers in this corpus:

```
>>> import nltk
>>> nltk.corpus.gutenberg.fileids()
['austen-emma.txt', 'austen-persuasion.txt', 'austen-sense.txt', 'bible-kjv.txt',
'blake-poems.txt', 'bryant-stories.txt', 'burgess-busterbrown.txt',
'carroll-alice.txt', 'chesterton-ball.txt', 'chesterton-brown.txt',
'chesterton-thursday.txt', 'edgeworth-parents.txt', 'melville-moby_dick.txt',
'milton-paradise.txt', 'shakespeare-caesar.txt', 'shakespeare-hamlet.txt',
'shakespeare-macbeth.txt', 'whitman-leaves.txt']
```

Let's pick out the first of these texts—*Emma* by Jane Austen—and give it a short name, `emma`, then find out how many words it contains:

```
>>> emma = nltk.corpus.gutenberg.words('austen-emma.txt')
>>> len(emma)
192427
```



In [Section 1.1](#), we showed how you could carry out concordancing of a text such as `text1` with the command `text1.concordance()`. However, this assumes that you are using one of the nine texts obtained as a result of doing `from nltk.book import *`. Now that you have started examining data from `nltk.corpus`, as in the previous example, you have to employ the following pair of statements to perform concordancing and other tasks from [Section 1.1](#):

```
>>> emma = nltk.Text(nltk.corpus.gutenberg.words('austen-emma.txt'))
>>> emma.concordance("surprise")
```

When we defined `emma`, we invoked the `words()` function of the `gutenberg` object in NLTK's `corpus` package. But since it is cumbersome to type such long names all the time, Python provides another version of the `import` statement, as follows:

```
>>> from nltk.corpus import gutenberg
>>> gutenberg.fileids()
['austen-emma.txt', 'austen-persuasion.txt', 'austen-sense.txt', ...]
>>> emma = gutenberg.words('austen-emma.txt')
```

Let's write a short program to display other information about each text, by looping over all the values of `fileid` corresponding to the `gutenberg` file identifiers listed earlier and then computing statistics for each text. For a compact output display, we will make sure that the numbers are all integers, using `int()`.

```
>>> for fileid in gutenberg.fileids():
...     num_chars = len(gutenberg.raw(fileid)) ❶
...     num_words = len(gutenberg.words(fileid))
...     num_sents = len(gutenberg.sents(fileid))
```

```

...     num_vocab = len(set([w.lower() for w in gutenbergl.words(fileid)]))
...     print int(num_chars/num_words), int(num_words/num_sents), int(num_words/num_vocab),
...         fileid
...
4 21 26 austen-emma.txt
4 23 16 austen-persuasion.txt
4 24 22 austen-sense.txt
4 33 79 bible-kjv.txt
4 18 5 blake-poems.txt
4 17 14 bryant-stories.txt
4 17 12 burgess-busterbrown.txt
4 16 12 carroll-alice.txt
4 17 11 chesterton-ball.txt
4 19 11 chesterton-brown.txt
4 16 10 chesterton-thursday.txt
4 18 24 edgeworth-parents.txt
4 24 15 melville-moby_dick.txt
4 52 10 milton-paradise.txt
4 12 8 shakespeare-caesar.txt
4 13 7 shakespeare-hamlet.txt
4 13 6 shakespeare-macbeth.txt
4 35 12 whitman-leaves.txt

```

This program displays three statistics for each text: average word length, average sentence length, and the number of times each vocabulary item appears in the text on average (our lexical diversity score). Observe that average word length appears to be a general property of English, since it has a recurrent value of 4. (In fact, the average word length is really 3, not 4, since the `num_chars` variable counts space characters.) By contrast average sentence length and lexical diversity appear to be characteristics of particular authors.

The previous example also showed how we can access the “raw” text of the book ❶, not split up into tokens. The `raw()` function gives us the contents of the file without any linguistic processing. So, for example, `len(gutenberg.raw('blake-poems.txt'))` tells us how many *letters* occur in the text, including the spaces between words. The `sents()` function divides the text up into its sentences, where each sentence is a list of words:

```

>>> macbeth_sentences = gutenberg.sents('shakespeare-macbeth.txt')
>>> macbeth_sentences
[['[', 'The', 'Tragedie', 'of', 'Macbeth', 'by', 'William', 'Shakespeare',
'1603', ''], ['Actus', 'Primus', '.'], ...]
>>> macbeth_sentences[1037]
['Double', ',', 'double', ',', 'toile', 'and', 'trouble', ';',
'Fire', 'burne', ',', 'and', 'Cauldron', 'bubble']
>>> longest_len = max([len(s) for s in macbeth_sentences])
>>> [s for s in macbeth_sentences if len(s) == longest_len]
[['Doubtfull', 'it', 'stood', ',', 'As', 'two', 'spent', 'Swimmers', ',', 'that',
'doe', 'cling', 'together', ',', 'And', 'choake', 'their', 'Art', ':', 'The',
'mercilesse', 'Macdonwald', ...], ...]

```



Most NLTK corpus readers include a variety of access methods apart from `words()`, `raw()`, and `sents()`. Richer linguistic content is available from some corpora, such as part-of-speech tags, dialogue tags, syntactic trees, and so forth; we will see these in later chapters.

Web and Chat Text

Although Project Gutenberg contains thousands of books, it represents established literature. It is important to consider less formal language as well. NLTK's small collection of web text includes content from a Firefox discussion forum, conversations overheard in New York, the movie script of *Pirates of the Carribean*, personal advertisements, and wine reviews:

```
>>> from nltk.corpus import webtext
>>> for fileid in webtext.fileids():
...     print fileid, webtext.raw(fileid)[:65], '...'
...
firefox.txt Cookie Manager: "Don't allow sites that set removed cookies to se...
grail.txt SCENE 1: [wind] [clap clap clap] KING ARTHUR: Whoa there! [clap...
overheard.txt White guy: So, do you have any plans for this evening? Asian girl...
pirates.txt PIRATES OF THE CARRIBEAN: DEAD MAN'S CHEST, by Ted Elliott & Terr...
singles.txt 25 SEXY MALE, seeks attrac older single lady, for discreet encoun...
wine.txt Lovely delicate, fragrant Rhone wine. Polished leather and strawb...
```

There is also a corpus of instant messaging chat sessions, originally collected by the Naval Postgraduate School for research on automatic detection of Internet predators. The corpus contains over 10,000 posts, anonymized by replacing usernames with generic names of the form “UserNNN”, and manually edited to remove any other identifying information. The corpus is organized into 15 files, where each file contains several hundred posts collected on a given date, for an age-specific chatroom (teens, 20s, 30s, 40s, plus a generic adults chatroom). The filename contains the date, chatroom, and number of posts; e.g., `10-19-20s_706posts.xml` contains 706 posts gathered from the 20s chat room on 10/19/2006.

```
>>> from nltk.corpus import nps_chat
>>> chatroom = nps_chat.posts('10-19-20s_706posts.xml')
>>> chatroom[123]
['i', 'do', 'n't', 'want', 'hot', 'pics', 'of', 'a', 'female', ',',
'I', 'can', 'look', 'in', 'a', 'mirror', '.']
```

Brown Corpus

The Brown Corpus was the first million-word electronic corpus of English, created in 1961 at Brown University. This corpus contains text from 500 sources, and the sources have been categorized by genre, such as *news*, *editorial*, and so on. Table 2-1 gives an example of each genre (for a complete list, see <http://icame.uib.no/brown/bcm-los.html>).

Table 2-1. Example document for each section of the Brown Corpus

ID	File	Genre	Description
A16	ca16	news	Chicago Tribune: <i>Society Reportage</i>
B02	cb02	editorial	Christian Science Monitor: <i>Editorials</i>
C17	cc17	reviews	Time Magazine: <i>Reviews</i>
D12	cd12	religion	Underwood: <i>Probing the Ethics of Realtors</i>
E36	ce36	hobbies	Norling: <i>Renting a Car in Europe</i>
F25	cf25	lore	Boroff: <i>Jewish Teenage Culture</i>
G22	cg22	belles_lettres	Reiner: <i>Coping with Runaway Technology</i>
H15	ch15	government	US Office of Civil and Defence Mobilization: <i>The Family Fallout Shelter</i>
J17	cj19	learned	Mosteller: <i>Probability with Statistical Applications</i>
K04	ck04	fiction	W.E.B. Du Bois: <i>Worlds of Color</i>
L13	cl13	mystery	Hitchens: <i>Footsteps in the Night</i>
M01	cm01	science_fiction	Heinlein: <i>Stranger in a Strange Land</i>
N14	cn15	adventure	Field: <i>Rattlesnake Ridge</i>
P12	cp12	romance	Callaghan: <i>A Passion in Rome</i>
R06	cr06	humor	Thurber: <i>The Future, If Any, of Comedy</i>

We can access the corpus as a list of words or a list of sentences (where each sentence is itself just a list of words). We can optionally specify particular categories or files to read:

```
>>> from nltk.corpus import brown
>>> brown.categories()
['adventure', 'belles_lettres', 'editorial', 'fiction', 'government', 'hobbies',
 'humor', 'learned', 'lore', 'mystery', 'news', 'religion', 'reviews', 'romance',
 'science_fiction']
>>> brown.words(categories='news')
['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', ...]
>>> brown.words(fileids=['cg22'])
['Does', 'our', 'society', 'have', 'a', 'runaway', ',', ...]
>>> brown.sents(categories=['news', 'editorial', 'reviews'])
[['The', 'Fulton', 'County'...], ['The', 'jury', 'further'...], ...]
```

The Brown Corpus is a convenient resource for studying systematic differences between genres, a kind of linguistic inquiry known as **stylistics**. Let's compare genres in their usage of modal verbs. The first step is to produce the counts for a particular genre. Remember to `import nltk` before doing the following:

```
>>> from nltk.corpus import brown
>>> news_text = brown.words(categories='news')
>>> fdist = nltk.FreqDist([w.lower() for w in news_text])
>>> modals = ['can', 'could', 'may', 'might', 'must', 'will']
>>> for m in modals:
...     print m + ': ', fdist[m],
```

```
...
can: 94 could: 87 may: 93 might: 38 must: 53 will: 389
```



Your Turn: Choose a different section of the Brown Corpus, and adapt the preceding example to count a selection of *wh* words, such as *what*, *when*, *where*, *who* and *why*.

Next, we need to obtain counts for each genre of interest. We'll use NLTK's support for conditional frequency distributions. These are presented systematically in [Section 2.2](#), where we also unpick the following code line by line. For the moment, you can ignore the details and just concentrate on the output.

```
>>> cfd = nltk.ConditionalFreqDist(
...     (genre, word)
...     for genre in brown.categories()
...     for word in brown.words(categories=genre))
>>> genres = ['news', 'religion', 'hobbies', 'science_fiction', 'romance', 'humor']
>>> modals = ['can', 'could', 'may', 'might', 'must', 'will']
>>> cfd.tabulate(conditions=genres, samples=modals)
```

	can	could	may	might	must	will
news	93	86	66	38	50	389
religion	82	59	78	12	54	71
hobbies	268	58	131	22	83	264
science_fiction	16	49	4	12	8	16
romance	74	193	11	51	45	43
humor	16	30	8	8	9	13

Observe that the most frequent modal in the news genre is *will*, while the most frequent modal in the romance genre is *could*. Would you have predicted this? The idea that word counts might distinguish genres will be taken up again in [Chapter 6](#).

Reuters Corpus

The Reuters Corpus contains 10,788 news documents totaling 1.3 million words. The documents have been classified into 90 topics, and grouped into two sets, called “training” and “test”; thus, the text with fileid ‘test/14826’ is a document drawn from the test set. This split is for training and testing algorithms that automatically detect the topic of a document, as we will see in [Chapter 6](#).

```
>>> from nltk.corpus import reuters
>>> reuters.fileids()
['test/14826', 'test/14828', 'test/14829', 'test/14832', ...]
>>> reuters.categories()
['acq', 'alum', 'barley', 'bop', 'carcass', 'castor-oil', 'cocoa',
'coconut', 'coconut-oil', 'coffee', 'copper', 'copra-cake', 'corn',
'cotton', 'cotton-oil', 'cpi', 'cpu', 'crude', 'dfl', 'dlr', ...]
```

Unlike the Brown Corpus, categories in the Reuters Corpus overlap with each other, simply because a news story often covers multiple topics. We can ask for the topics

covered by one or more documents, or for the documents included in one or more categories. For convenience, the corpus methods accept a single fileid or a list of fileids.

```
>>> reuters.categories('training/9865')
['barley', 'corn', 'grain', 'wheat']
>>> reuters.categories(['training/9865', 'training/9880'])
['barley', 'corn', 'grain', 'money-fx', 'wheat']
>>> reuters.fileids('barley')
['test/15618', 'test/15649', 'test/15676', 'test/15728', 'test/15871', ...]
>>> reuters.fileids(['barley', 'corn'])
['test/14832', 'test/14858', 'test/15033', 'test/15043', 'test/15106',
'test/15287', 'test/15341', 'test/15618', 'test/15618', 'test/15648', ...]
```

Similarly, we can specify the words or sentences we want in terms of files or categories. The first handful of words in each of these texts are the titles, which by convention are stored as uppercase.

```
>>> reuters.words('training/9865')[:14]
['FRENCH', 'FREE', 'MARKET', 'CEREAL', 'EXPORT', 'BIDS',
'DETAILED', 'French', 'operators', 'have', 'requested', 'licences', 'to', 'export']
>>> reuters.words(['training/9865', 'training/9880'])
['FRENCH', 'FREE', 'MARKET', 'CEREAL', 'EXPORT', ...]
>>> reuters.words(categories='barley')
['FRENCH', 'FREE', 'MARKET', 'CEREAL', 'EXPORT', ...]
>>> reuters.words(categories=['barley', 'corn'])
['THAI', 'TRADE', 'DEFICIT', 'WIDENS', 'IN', 'FIRST', ...]
```

Inaugural Address Corpus

In [Section 1.1](#), we looked at the Inaugural Address Corpus, but treated it as a single text. The graph in [Figure 1-2](#) used “word offset” as one of the axes; this is the numerical index of the word in the corpus, counting from the first word of the first address. However, the corpus is actually a collection of 55 texts, one for each presidential address. An interesting property of this collection is its time dimension:

```
>>> from nltk.corpus import inaugural
>>> inaugural.fileids()
['1789-Washington.txt', '1793-Washington.txt', '1797-Adams.txt', ...]
>>> [fileid[:4] for fileid in inaugural.fileids()]
['1789', '1793', '1797', '1801', '1805', '1809', '1813', '1817', '1821', ...]
```

Notice that the year of each text appears in its filename. To get the year out of the filename, we extracted the first four characters, using `fileid[:4]`.

Let’s look at how the words *America* and *citizen* are used over time. The following code converts the words in the Inaugural corpus to lowercase using `w.lower()` ❶, then checks whether they start with either of the “targets” *america* or *citizen* using `startswith()` ❶. Thus it will count words such as *American’s* and *Citizens*. We’ll learn about conditional frequency distributions in [Section 2.2](#); for now, just consider the output, shown in [Figure 2-1](#).

```
>>> cfd = nltk.ConditionalFreqDist(
...     (target, file[:4])
...     for fileid in inaugural.fileids()
...     for w in inaugural.words(fileid)
...     for target in ['america', 'citizen']
...     if w.lower().startswith(target)) ❶
>>> cfd.plot()
```

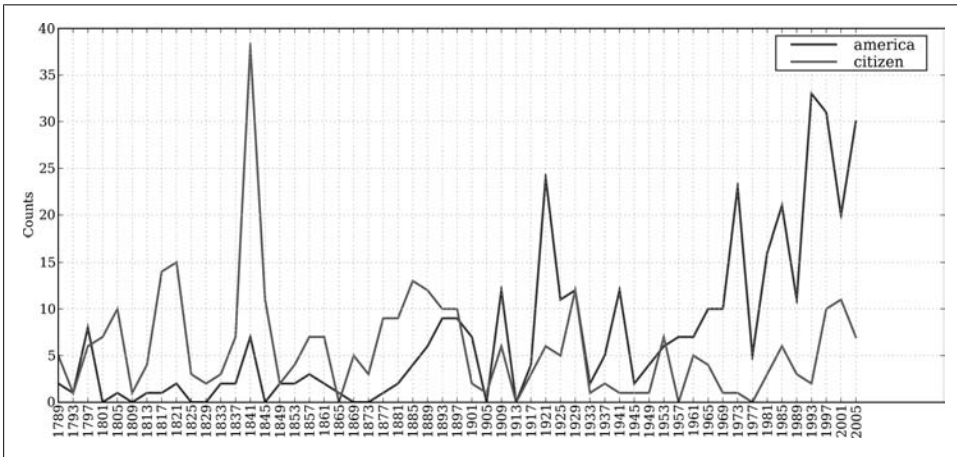


Figure 2-1. Plot of a conditional frequency distribution: All words in the Inaugural Address Corpus that begin with *america* or *citizen* are counted; separate counts are kept for each address; these are plotted so that trends in usage over time can be observed; counts are not normalized for document length.

Annotated Text Corpora

Many text corpora contain linguistic annotations, representing part-of-speech tags, named entities, syntactic structures, semantic roles, and so forth. NLTK provides convenient ways to access several of these corpora, and has data packages containing corpora and corpus samples, freely downloadable for use in teaching and research. Table 2-2 lists some of the corpora. For information about downloading them, see <http://www.nltk.org/data>. For more examples of how to access NLTK corpora, please consult the Corpus HOWTO at <http://www.nltk.org/howto>.

Table 2-2. Some of the corpora and corpus samples distributed with NLTK

Corpus	Compiler	Contents
Brown Corpus	Francis, Kucera	15 genres, 1.15M words, tagged, categorized
CESS Treebanks	CLiC-UB	1M words, tagged and parsed (Catalan, Spanish)
Chat-80 Data Files	Pereira & Warren	World Geographic Database
CMU Pronouncing Dictionary	CMU	127k entries
CoNLL 2000 Chunking Data	CoNLL	270k words, tagged and chunked

Corpus	Compiler	Contents
CoNLL 2002 Named Entity	CoNLL	700k words, POS and named entity tagged (Dutch, Spanish)
CoNLL 2007 Dependency Parsed Treebanks (selections)	CoNLL	150k words, dependency parsed (Basque, Catalan)
Dependency Treebank	Narad	Dependency parsed version of Penn Treebank sample
Floresta Treebank	Diana Santos et al.	9k sentences, tagged and parsed (Portuguese)
Gazetteer Lists	Various	Lists of cities and countries
Genesis Corpus	Misc web sources	6 texts, 200k words, 6 languages
Gutenberg (selections)	Hart, Newby, et al.	18 texts, 2M words
Inaugural Address Corpus	Cspan	U.S. Presidential Inaugural Addresses (1789–present)
Indian POS Tagged Corpus	Kumaran et al.	60k words, tagged (Bangla, Hindi, Marathi, Telugu)
MacMorpho Corpus	NILC, USP, Brazil	1M words, tagged (Brazilian Portuguese)
Movie Reviews	Pang, Lee	2k movie reviews with sentiment polarity classification
Names Corpus	Kantrowitz, Ross	8k male and female names
NIST 1999 Info Extr (selections)	Garofolo	63k words, newswire and named entity SGML markup
NPS Chat Corpus	Forsyth, Martell	10k IM chat posts, POS and dialogue-act tagged
Penn Treebank (selections)	LDC	40k words, tagged and parsed
PP Attachment Corpus	Ratnaparkhi	28k prepositional phrases, tagged as noun or verb modifiers
Proposition Bank	Palmer	113k propositions, 3,300 verb frames
Question Classification	Li, Roth	6k questions, categorized
Reuters Corpus	Reuters	1.3M words, 10k news documents, categorized
Roget's Thesaurus	Project Gutenberg	200k words, formatted text
RTE Textual Entailment	Dagan et al.	8k sentence pairs, categorized
SEMCOR	Rus, Mihalcea	880k words, POS and sense tagged
Senseval 2 Corpus	Pedersen	600k words, POS and sense tagged
Shakespeare texts (selections)	Bosak	8 books in XML format
State of the Union Corpus	Cspan	485k words, formatted text
Stopwords Corpus	Porter et al.	2,400 stopwords for 11 languages
Swadesh Corpus	Wiktionary	Comparative wordlists in 24 languages
Switchboard Corpus (selections)	LDC	36 phone calls, transcribed, parsed
TIMIT Corpus (selections)	NIST/LDC	Audio files and transcripts for 16 speakers
Univ Decl of Human Rights	United Nations	480k words, 300+ languages
VerbNet 2.1	Palmer et al.	5k verbs, hierarchically organized, linked to WordNet
Wordlist Corpus	OpenOffice.org et al.	960k words and 20k affixes for 8 languages
WordNet 3.0 (English)	Miller, Fellbaum	145k synonym sets

Corpora in Other Languages

NLTK comes with corpora for many languages, though in some cases you will need to learn how to manipulate character encodings in Python before using these corpora (see [Section 3.3](#)).

```
>>> nltk.corpus.cess_esp.words()
['El', 'grupo', 'estatal', 'Electricit\xe9_de_France', ...]
>>> nltk.corpus.floresta.words()
['Um', 'revivalismo', 'refrescante', '0', '7_e_Meio', ...]
>>> nltk.corpus.indian.words('hindi.pos')
['\xe0\xa4\xaa\xe0\xa5\x82\xe0\xa4\xb0\xe0\xa5\x8d\xe0\xa4\xa3',
 '\xe0\xa4\xaa\xe0\xa5\x8d\xe0\xa4\xb0\xe0\xa4\xa4\xe0\xa4\xbf\xe0\xa4\xac\xe0\xa4
 \x82\xe0\xa4\xa7', ...]
>>> nltk.corpus.udhr.fileids()
['Abkhaz-Cyrillic+Abkh', 'Abkhaz-UTF8', 'Achehnese-Latin1', 'Achuar-Shiwiar-Latin1',
 'Adja-UTF8', 'Afaan_Oromo_Oromiffa-Latin1', 'Afrikaans-Latin1', 'Aguaruna-Latin1',
 'Akuapem-Twi-UTF8', 'Albanian-Shqip-Latin1', 'Amahuaca', 'Amahuaca-Latin1', ...]
>>> nltk.corpus.udhr.words('Javanese-Latin1')[11:]
[u'Saben', u'umat', u'manungsa', u'lair', u'kanthi', ...]
```

The last of these corpora, `udhr`, contains the Universal Declaration of Human Rights in over 300 languages. The `fileids` for this corpus include information about the character encoding used in the file, such as `UTF8` or `Latin1`. Let's use a conditional frequency distribution to examine the differences in word lengths for a selection of languages included in the `udhr` corpus. The output is shown in [Figure 2-2](#) (run the program yourself to see a color plot). Note that `True` and `False` are Python's built-in Boolean values.

```
>>> from nltk.corpus import udhr
>>> languages = ['Chickasaw', 'English', 'German_Deutsch',
...             'Greenlandic_Inuktitut', 'Hungarian_Magyar', 'Ibibio_Efik']
>>> cfd = nltk.ConditionalFreqDist(
...     (lang, len(word))
...     for lang in languages
...     for word in udhr.words(lang + '-Latin1'))
>>> cfd.plot(cumulative=True)
```



Your Turn: Pick a language of interest in `udhr.fileids()`, and define a variable `raw_text = udhr.raw(Language-Latin1)`. Now plot a frequency distribution of the letters of the text using

```
nltk.FreqDist(raw_text).plot().
```

Unfortunately, for many languages, substantial corpora are not yet available. Often there is insufficient government or industrial support for developing language resources, and individual efforts are piecemeal and hard to discover or reuse. Some languages have no established writing system, or are endangered. (See [Section 2.7](#) for suggestions on how to locate language resources.)

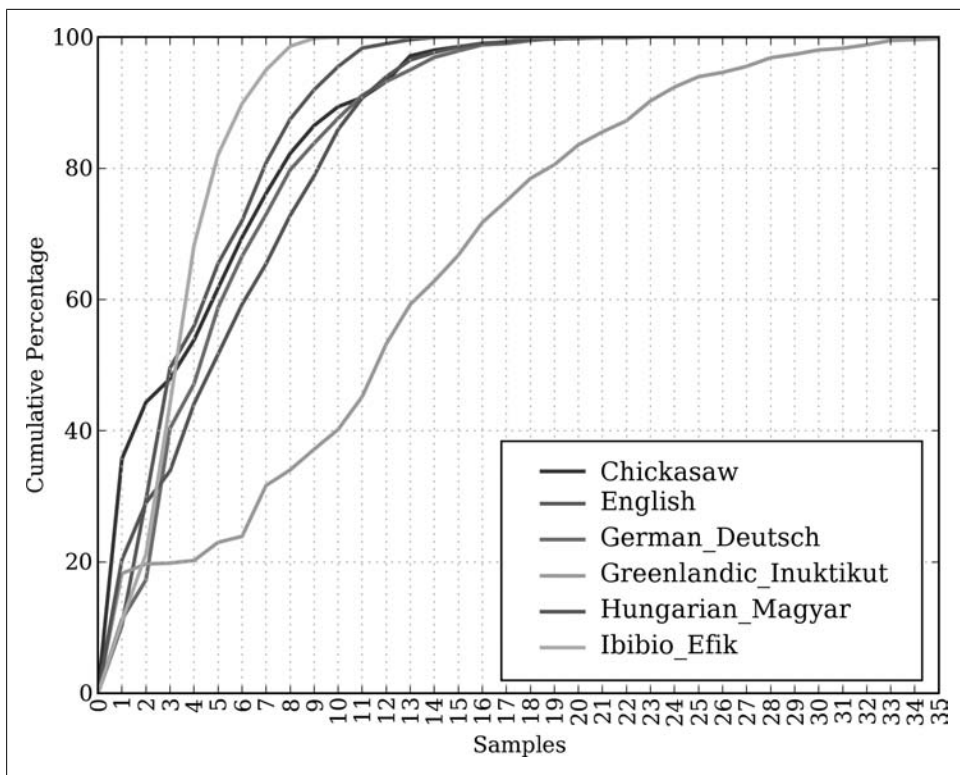


Figure 2-2. Cumulative word length distributions: Six translations of the Universal Declaration of Human Rights are processed; this graph shows that words having five or fewer letters account for about 80% of Ibibio text, 60% of German text, and 25% of Inuktitut text.

Text Corpus Structure

We have seen a variety of corpus structures so far; these are summarized in [Figure 2-3](#). The simplest kind lacks any structure: it is just a collection of texts. Often, texts are grouped into categories that might correspond to genre, source, author, language, etc. Sometimes these categories overlap, notably in the case of topical categories, as a text can be relevant to more than one topic. Occasionally, text collections have temporal structure, news collections being the most common example.

NLTK's corpus readers support efficient access to a variety of corpora, and can be used to work with new corpora. [Table 2-3](#) lists functionality provided by the corpus readers.

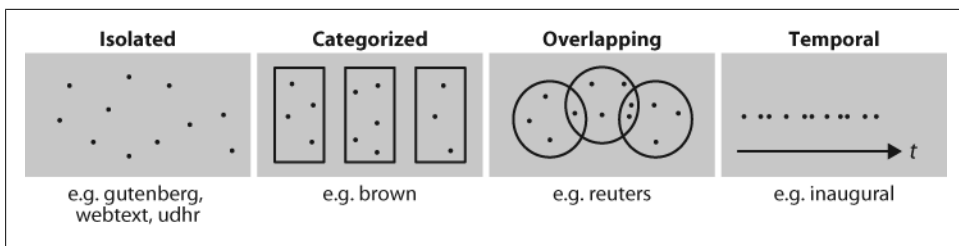


Figure 2-3. Common structures for text corpora: The simplest kind of corpus is a collection of isolated texts with no particular organization; some corpora are structured into categories, such as genre (Brown Corpus); some categorizations overlap, such as topic categories (Reuters Corpus); other corpora represent language use over time (Inaugural Address Corpus).

Table 2-3. Basic corpus functionality defined in NLTK: More documentation can be found using `help(nltk.corpus.reader)` and by reading the online Corpus HOWTO at <http://www.nltk.org/howto>.

Example	Description
<code>fileids()</code>	The files of the corpus
<code>fileids([categories])</code>	The files of the corpus corresponding to these categories
<code>categories()</code>	The categories of the corpus
<code>categories([fileids])</code>	The categories of the corpus corresponding to these files
<code>raw()</code>	The raw content of the corpus
<code>raw(fileids=[f1,f2,f3])</code>	The raw content of the specified files
<code>raw(categories=[c1,c2])</code>	The raw content of the specified categories
<code>words()</code>	The words of the whole corpus
<code>words(fileids=[f1,f2,f3])</code>	The words of the specified fileids
<code>words(categories=[c1,c2])</code>	The words of the specified categories
<code>sents()</code>	The sentences of the specified categories
<code>sents(fileids=[f1,f2,f3])</code>	The sentences of the specified fileids
<code>sents(categories=[c1,c2])</code>	The sentences of the specified categories
<code>abspath(fileid)</code>	The location of the given file on disk
<code>encoding(fileid)</code>	The encoding of the file (if known)
<code>open(fileid)</code>	Open a stream for reading the given corpus file
<code>root()</code>	The path to the root of locally installed corpus
<code>readme()</code>	The contents of the README file of the corpus

We illustrate the difference between some of the corpus access methods here:

```
>>> raw = gutenberg.raw("burgess-busterbrown.txt")
>>> raw[1:20]
'The Adventures of B'
>>> words = gutenberg.words("burgess-busterbrown.txt")
>>> words[1:20]
```

```
['The', 'Adventures', 'of', 'Buster', 'Bear', 'by', 'Thornton', 'W', '.',
'Burgess', '1920', ''], 'I', 'BUSTER', 'BEAR', 'GOES', 'FISHING', 'Buster',
'Bear']
>>> sents = gutenberg.sents("burgess-busterbrown.txt")
>>> sents[1:20]
[['I'], ['BUSTER', 'BEAR', 'GOES', 'FISHING'], ['Buster', 'Bear', 'yawned', 'as',
'he', 'lay', 'on', 'his', 'comfortable', 'bed', 'of', 'leaves', 'and', 'watched',
'the', 'first', 'early', 'morning', 'sunbeams', 'creeping', 'through', ...], ...]
```

Loading Your Own Corpus

If you have a your own collection of text files that you would like to access using the methods discussed earlier, you can easily load them with the help of NLTK's `Plain textCorpusReader`. Check the location of your files on your file system; in the following example, we have taken this to be the directory `/usr/share/dict`. Whatever the location, set this to be the value of `corpus_root` ❶. The second parameter of the `PlaintextCorpusReader` initializer ❷ can be a list of fileids, like `['a.txt', 'test/b.txt']`, or a pattern that matches all fileids, like `'[abc]/.*\.txt'` (see [Section 3.4](#) for information about regular expressions).

```
>>> from nltk.corpus import PlaintextCorpusReader
>>> corpus_root = '/usr/share/dict' ❶
>>> wordlists = PlaintextCorpusReader(corpus_root, '.*') ❷
>>> wordlists.fileids()
['README', 'connectives', 'propernames', 'web2', 'web2a', 'words']
>>> wordlists.words('connectives')
['the', 'of', 'and', 'to', 'a', 'in', 'that', 'is', ...]
```

As another example, suppose you have your own local copy of Penn Treebank (release 3), in `C:\corpora`. We can use the `BracketParseCorpusReader` to access this corpus. We specify the `corpus_root` to be the location of the parsed *Wall Street Journal* component of the corpus ❶, and give a `file_pattern` that matches the files contained within its subfolders ❷ (using forward slashes).

```
>>> from nltk.corpus import BracketParseCorpusReader
>>> corpus_root = r"C:\corpora\penntreebank\parsed\mrg\wsj" ❶
>>> file_pattern = r".*/wsj_.*\.mrg" ❷
>>> ptb = BracketParseCorpusReader(corpus_root, file_pattern)
>>> ptb.fileids()
['00/ws_j_0001.mrg', '00/ws_j_0002.mrg', '00/ws_j_0003.mrg', '00/ws_j_0004.mrg', ...]
>>> len(ptb.sents())
49208
>>> ptb.sents(fileids='20/ws_j_2013.mrg')[19]
['The', '55-year-old', 'Mr.', 'Noriega', 'is', 'n't', 'as', 'smooth', 'as', 'the',
'shah', 'of', 'Iran', ',', 'as', 'well-born', 'as', 'Nicaragua', "'s", 'Anastasio',
'Somoza', ',', 'as', 'imperial', 'as', 'Ferdinand', 'Marcos', 'of', 'the', 'Philippines',
'or', 'as', 'bloody', 'as', 'Haiti', "'s", 'Baby', 'Doc', 'Duvalier', '.']
```

2.2 Conditional Frequency Distributions

We introduced frequency distributions in [Section 1.3](#). We saw that given some list `mylist` of words or other items, `FreqDist(mylist)` would compute the number of occurrences of each item in the list. Here we will generalize this idea.

When the texts of a corpus are divided into several categories (by genre, topic, author, etc.), we can maintain separate frequency distributions for each category. This will allow us to study systematic differences between the categories. In the previous section, we achieved this using NLTK's `ConditionalFreqDist` data type. A **conditional frequency distribution** is a collection of frequency distributions, each one for a different “condition.” The condition will often be the category of the text. [Figure 2-4](#) depicts a fragment of a conditional frequency distribution having just two conditions, one for news text and one for romance text.

Condition: News		Condition: Romance	
the		the	
cute		cute	
Monday		Monday	
could		could	
will		will	

Figure 2-4. Counting words appearing in a text collection (a conditional frequency distribution).

Conditions and Events

A frequency distribution counts observable events, such as the appearance of words in a text. A conditional frequency distribution needs to pair each event with a condition. So instead of processing a sequence of words ❶, we have to process a sequence of pairs ❷:

```
>>> text = ['The', 'Fulton', 'County', 'Grand', 'Jury', 'said', ...] ❶
>>> pairs = [('news', 'The'), ('news', 'Fulton'), ('news', 'County'), ...] ❷
```

Each pair has the form *(condition, event)*. If we were processing the entire Brown Corpus by genre, there would be 15 conditions (one per genre) and 1,161,192 events (one per word).

Counting Words by Genre

In [Section 2.1](#), we saw a conditional frequency distribution where the condition was the section of the Brown Corpus, and for each condition we counted words. Whereas `FreqDist()` takes a simple list as input, `ConditionalFreqDist()` takes a list of pairs.

```
>>> from nltk.corpus import brown
>>> cfd = nltk.ConditionalFreqDist(
...     (genre, word)
...     for genre in brown.categories()
...     for word in brown.words(categories=genre))
```

Let's break this down, and look at just two genres, news and romance. For each genre ❷, we loop over every word in the genre ❸, producing pairs consisting of the genre and the word ❶:

```
>>> genre_word = [(genre, word) ❶
...               for genre in ['news', 'romance'] ❷
...               for word in brown.words(categories=genre)] ❸
>>> len(genre_word)
170576
```

So, as we can see in the following code, pairs at the beginning of the list `genre_word` will be of the form `('news', word)` ❶, whereas those at the end will be of the form `('romance', word)` ❷.

```
>>> genre_word[:4]
[('news', 'The'), ('news', 'Fulton'), ('news', 'County'), ('news', 'Grand')] ❶
>>> genre_word[-4:]
[('romance', 'afraid'), ('romance', 'not'), ('romance', ''), ('romance', '.')] ❷
```

We can now use this list of pairs to create a `ConditionalFreqDist`, and save it in a variable `cfd`. As usual, we can type the name of the variable to inspect it ❶, and verify it has two conditions ❷:

```
>>> cfd = nltk.ConditionalFreqDist(genre_word)
>>> cfd ❶
<ConditionalFreqDist with 2 conditions>
>>> cfd.conditions()
['news', 'romance'] ❷
```

Let's access the two conditions, and satisfy ourselves that each is just a frequency distribution:

```
>>> cfd['news']
<FreqDist with 100554 outcomes>
>>> cfd['romance']
<FreqDist with 70022 outcomes>
>>> list(cfd['romance'])
['.', '!', 'the', 'and', 'to', 'a', 'of', '``', '""', 'was', 'I', 'in', 'he', 'had',
 '?', 'her', 'that', 'it', 'his', 'she', 'with', 'you', 'for', 'at', 'He', 'on', 'him',
 'said', '!', '--', 'be', 'as', ';', 'have', 'but', 'not', 'would', 'She', 'The', ...]
>>> cfd['romance']['could']
193
```

Plotting and Tabulating Distributions

Apart from combining two or more frequency distributions, and being easy to initialize, a `ConditionalFreqDist` provides some useful methods for tabulation and plotting.

The plot in [Figure 2-1](#) was based on a conditional frequency distribution reproduced in the following code. The condition is either of the words *america* or *citizen* ❷, and the counts being plotted are the number of times the word occurred in a particular speech. It exploits the fact that the filename for each speech—for example, *1865-Lincoln.txt*—contains the year as the first four characters ❶. This code generates the pair ('america', '1865') for every instance of a word whose lowercased form starts with *america*—such as *Americans*—in the file *1865-Lincoln.txt*.

```
>>> from nltk.corpus import inaugural
>>> cfd = nltk.ConditionalFreqDist(
...     (target, fileid[:4]) ❶
...     for fileid in inaugural.fileids()
...     for w in inaugural.words(fileid)
...     for target in ['america', 'citizen'] ❷
...     if w.lower().startswith(target))
```

The plot in [Figure 2-2](#) was also based on a conditional frequency distribution, reproduced in the following code. This time, the condition is the name of the language, and the counts being plotted are derived from word lengths ❶. It exploits the fact that the filename for each language is the language name followed by '-Latin1' (the character encoding).

```
>>> from nltk.corpus import udhr
>>> languages = ['Chickasaw', 'English', 'German_Deutsch',
...             'Greenlandic_Inuktitut', 'Hungarian_Magyar', 'Ibibio_Efik']
>>> cfd = nltk.ConditionalFreqDist(
...     (lang, len(word)) ❶
...     for lang in languages
...     for word in udhr.words(lang + '-Latin1'))
```

In the `plot()` and `tabulate()` methods, we can optionally specify which conditions to display with a `conditions=` parameter. When we omit it, we get all the conditions. Similarly, we can limit the samples to display with a `samples=` parameter. This makes it possible to load a large quantity of data into a conditional frequency distribution, and then to explore it by plotting or tabulating selected conditions and samples. It also gives us full control over the order of conditions and samples in any displays. For example, we can tabulate the cumulative frequency data just for two languages, and for words less than 10 characters long, as shown next. We interpret the last cell on the top row to mean that 1,638 words of the English text have nine or fewer letters.

```
>>> cfd.tabulate(conditions=['English', 'German_Deutsch'],
...             samples=range(10), cumulative=True)
...           0   1   2   3   4   5   6   7   8   9
English      0  185  525  883  997 1166 1283 1440 1558 1638
German_Deutsch 0  171  263  614  717  894 1013 1110 1213 1275
```




Your Turn: Working with the news and romance genres from the Brown Corpus, find out which days of the week are most newsworthy, and which are most romantic. Define a variable called `days` containing a list of days of the week, i.e., `['Monday', ...]`. Now tabulate the counts for these words using `cfld.tabulate(samples=days)`. Now try the same thing using `plot` in place of `tabulate`. You may control the output order of days with the help of an extra parameter: `conditions=['Monday', ...]`.

You may have noticed that the multiline expressions we have been using with conditional frequency distributions look like list comprehensions, but without the brackets. In general, when we use a list comprehension as a parameter to a function, like `set([w.lower for w in t])`, we are permitted to omit the square brackets and just write `set(w.lower() for w in t)`. (See the discussion of “generator expressions” in [Section 4.2](#) for more about this.)

Generating Random Text with Bigrams

We can use a conditional frequency distribution to create a table of bigrams (word pairs, introduced in [Section 1.3](#)). The `bigrams()` function takes a list of words and builds a list of consecutive word pairs:

```
>>> sent = ['In', 'the', 'beginning', 'God', 'created', 'the', 'heaven',
...         'and', 'the', 'earth', '.']
>>> nltk.bigrams(sent)
[('In', 'the'), ('the', 'beginning'), ('beginning', 'God'), ('God', 'created'),
 ('created', 'the'), ('the', 'heaven'), ('heaven', 'and'), ('and', 'the'),
 ('the', 'earth'), ('earth', '.')]
```

In [Example 2-1](#), we treat each word as a condition, and for each one we effectively create a frequency distribution over the following words. The function `generate_model()` contains a simple loop to generate text. When we call the function, we choose a word (such as `'living'`) as our initial context. Then, once inside the loop, we print the current value of the variable `word`, and reset `word` to be the most likely token in that context (using `max()`); next time through the loop, we use that word as our new context. As you can see by inspecting the output, this simple approach to text generation tends to get stuck in loops. Another method would be to randomly choose the next word from among the available words.

Example 2-1. Generating random text: This program obtains all bigrams from the text of the book of Genesis, then constructs a conditional frequency distribution to record which words are most likely to follow a given word; e.g., after the word `living`, the most likely word is `creature`; the `generate_model()` function uses this data, and a seed word, to generate random text.

```
def generate_model(cfldist, word, num=15):
    for i in range(num):
        print word,
        word = cfldist[word].max()
```

```
text = nltk.corpus.genesis.words('english-kjv.txt')
bigrams = nltk.bigrams(text)
cfd = nltk.ConditionalFreqDist(bigrams) ❶

>>> print cfd['living']
<FreqDist: 'creature': 7, 'thing': 4, 'substance': 2, ',': 1, '.': 1, 'soul': 1>
>>> generate_model(cfd, 'living')
living creature that he said , and the land of the land of the land
```

Conditional frequency distributions are a useful data structure for many NLP tasks. Their commonly used methods are summarized in [Table 2-4](#).

Table 2-4. NLTK’s conditional frequency distributions: Commonly used methods and idioms for defining, accessing, and visualizing a conditional frequency distribution of counters

Example	Description
<code>cfdist = ConditionalFreqDist(pairs)</code>	Create a conditional frequency distribution from a list of pairs
<code>cfdist.conditions()</code>	Alphabetically sorted list of conditions
<code>cfdist[condition]</code>	The frequency distribution for this condition
<code>cfdist[condition][sample]</code>	Frequency for the given sample for this condition
<code>cfdist.tabulate()</code>	Tabulate the conditional frequency distribution
<code>cfdist.tabulate(samples, conditions)</code>	Tabulation limited to the specified samples and conditions
<code>cfdist.plot()</code>	Graphical plot of the conditional frequency distribution
<code>cfdist.plot(samples, conditions)</code>	Graphical plot limited to the specified samples and conditions
<code>cfdist1 < cfdist2</code>	Test if samples in <code>cfdist1</code> occur less frequently than in <code>cfdist2</code>

2.3 More Python: Reusing Code

By this time you’ve probably typed and retyped a lot of code in the Python interactive interpreter. If you mess up when retyping a complex example, you have to enter it again. Using the arrow keys to access and modify previous commands is helpful but only goes so far. In this section, we see two important ways to reuse code: text editors and Python functions.

Creating Programs with a Text Editor

The Python interactive interpreter performs your instructions as soon as you type them. Often, it is better to compose a multiline program using a text editor, then ask Python to run the whole program at once. Using IDLE, you can do this by going to the File menu and opening a new window. Try this now, and enter the following one-line program:

```
print 'Monty Python'
```

Save this program in a file called *monty.py*, then go to the Run menu and select the command Run Module. (We'll learn what modules are shortly.) The result in the main IDLE window should look like this:

```
>>> ===== RESTART =====
>>>
Monty Python
>>>
```

You can also type `from monty import *` and it will do the same thing.

From now on, you have a choice of using the interactive interpreter or a text editor to create your programs. It is often convenient to test your ideas using the interpreter, revising a line of code until it does what you expect. Once you're ready, you can paste the code (minus any `>>>` or `...` prompts) into the text editor, continue to expand it, and finally save the program in a file so that you don't have to type it in again later. Give the file a short but descriptive name, using all lowercase letters and separating words with underscore, and using the *.py* filename extension, e.g., *monty_python.py*.



Important: Our inline code examples include the `>>>` and `...` prompts as if we are interacting directly with the interpreter. As they get more complicated, you should instead type them into the editor, without the prompts, and run them from the editor as shown earlier. When we provide longer programs in this book, we will leave out the prompts to remind you to type them into a file rather than using the interpreter. You can see this already in [Example 2-1](#). Note that the example still includes a couple of lines with the Python prompt; this is the interactive part of the task where you inspect some data and invoke a function. Remember that all code samples like [Example 2-1](#) are downloadable from <http://www.nltk.org/>.

Functions

Suppose that you work on analyzing text that involves different forms of the same word, and that part of your program needs to work out the plural form of a given singular noun. Suppose it needs to do this work in two places, once when it is processing some texts and again when it is processing user input.

Rather than repeating the same code several times over, it is more efficient and reliable to localize this work inside a **function**. A function is just a named block of code that performs some well-defined task, as we saw in [Section 1.1](#). A function is usually defined to take some inputs, using special variables known as **parameters**, and it may produce a result, also known as a **return value**. We define a function using the keyword `def` followed by the function name and any input parameters, followed by the body of the function. Here's the function we saw in [Section 1.1](#) (including the `import` statement that makes division behave as expected):

```
>>> from __future__ import division
>>> def lexical_diversity(text):
...     return len(text) / len(set(text))
```

We use the keyword `return` to indicate the value that is produced as output by the function. In this example, all the work of the function is done in the `return` statement. Here’s an equivalent definition that does the same work using multiple lines of code. We’ll change the parameter name from `text` to `my_text_data` to remind you that this is an arbitrary choice:

```
>>> def lexical_diversity(my_text_data):
...     word_count = len(my_text_data)
...     vocab_size = len(set(my_text_data))
...     diversity_score = word_count / vocab_size
...     return diversity_score
```

Notice that we’ve created some new variables inside the body of the function. These are **local variables** and are not accessible outside the function. So now we have defined a function with the name `lexical_diversity`. But just defining it won’t produce any output! Functions do nothing until they are “called” (or “invoked”).

Let’s return to our earlier scenario, and actually define a simple function to work out English plurals. The function `plural()` in [Example 2-2](#) takes a singular noun and generates a plural form, though it is not always correct. (We’ll discuss functions at greater length in [Section 4.4](#).)

Example 2-2. A Python function: This function tries to work out the plural form of any English noun; the keyword `def` (define) is followed by the function name, then a parameter inside parentheses, and a colon; the body of the function is the indented block of code; it tries to recognize patterns within the word and process the word accordingly; e.g., if the word ends with `y`, delete the `y` and add `ies`.

```
def plural(word):
    if word.endswith('y'):
        return word[:-1] + 'ies'
    elif word[-1] in 'sx' or word[-2:] in ['sh', 'ch']:
        return word + 'es'
    elif word.endswith('an'):
        return word[:-2] + 'en'
    else:
        return word + 's'

>>> plural('fairy')
'fairies'
>>> plural('woman')
'women'
```

The `endswith()` function is always associated with a string object (e.g., `word` in [Example 2-2](#)). To call such functions, we give the name of the object, a period, and then the name of the function. These functions are usually known as **methods**.

Modules

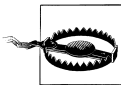
Over time you will find that you create a variety of useful little text-processing functions, and you end up copying them from old programs to new ones. Which file contains the latest version of the function you want to use? It makes life a lot easier if you can collect your work into a single place, and access previously defined functions without making copies.

To do this, save your function(s) in a file called (say) *textproc.py*. Now, you can access your work simply by importing it from the file:

```
>>> from textproc import plural
>>> plural('wish')
wishes
>>> plural('fan')
fen
```

Our plural function obviously has an error, since the plural of *fan* is *fans*. Instead of typing in a new version of the function, we can simply edit the existing one. Thus, at every stage, there is only one version of our plural function, and no confusion about which one is being used.

A collection of variable and function definitions in a file is called a Python **module**. A collection of related modules is called a **package**. NLTK's code for processing the Brown Corpus is an example of a module, and its collection of code for processing all the different corpora is an example of a package. NLTK itself is a set of packages, sometimes called a **library**.



Caution!

If you are creating a file to contain some of your Python code, do *not* name your file *nltk.py*: it may get imported in place of the “real” NLTK package. When it imports modules, Python first looks in the current directory (folder).

2.4 Lexical Resources

A lexicon, or lexical resource, is a collection of words and/or phrases along with associated information, such as part-of-speech and sense definitions. Lexical resources are secondary to texts, and are usually created and enriched with the help of texts. For example, if we have defined a text `my_text`, then `vocab = sorted(set(my_text))` builds the vocabulary of `my_text`, whereas `word_freq = FreqDist(my_text)` counts the frequency of each word in the text. Both `vocab` and `word_freq` are simple lexical resources. Similarly, a concordance like the one we saw in [Section 1.1](#) gives us information about word usage that might help in the preparation of a dictionary. Standard terminology for lexicons is illustrated in [Figure 2-5](#). A **lexical entry** consists of a **headword** (also known as a **lemma**) along with additional information, such as the part-of-speech and